↳ Extracting Information

Let's move to another example - an extraction task, which a generative model can do very well. Given context, which in this case is a description of a movie, we want the model to extract the movie title.

```
generate_text("""
    Extract the movie title from the text below.
    Deadpool 2 | Official HD Deadpool's "Wet on Wet"
    Teaser | 2018 """)
```

O/P # Response

Deadpool 2

↳ Rewriting Text

The model is also effective at tasks that involve taking a piece of text & rewriting it into another format that we need.

Here's an example. We have a one-line instruction followed by the context, which in this case is a blog excerpt. The instruction is to generate a list of frequently asked questions (FAQ) based on the passage, which involves a mixture of several tasks, such as extraction & rewriting.

generate_text ("""
Given the following text, write down a list of potential FAQs; together with the answers.
The Cohere Platform provides an API for developers & organizations to access cutting edge LLMs w/o needing machine learning know-how.

- - -

- -

O/P #Response

Here is a list of potential FAQs based on provided text:

- **Q: What does Cohere Platform offer? -
  A: - -

-

↳ By now, we can see how versatile our model is at performing various forms of tasks - not just free form text generation, but also following instructions, working with contextual info, summarizing long passages, extracting info, rewriting text into diff. formats & more.

↳ This is just a taste of what kinds of prompts you can design. You can keep layering your instructions to be as specific as you want & see the O/P generated by the model. And there is really no right or wrong way to design a prompt. It's really about applying an idea & continuing to iterate the prompt until you get the outcome you are looking for.

Fine-Tuning for Chat

Here, you will learn how to fine-tune the chat endpoint models on custom datasets, ~~enabling~~ enhancing its performance on specific tasks.

↳ Cohere's LLMs have been trained to be useful in practical business app's. Using the Chat endpoint, you can leverage a pre-trained LLM to build a chatbot that performs tasks ranging from summarization to copywriting to question answering.

↳ Depending on the use case you have in mind, you might want to amend how the chatbot generates its output. For instance, if your task uses highly technical data or you want to change the chatbot's o/p format, you often need to perform another round of training on additional data to ensure the best performance. This extra training is referred to as fine-tuning.

↳ Fine-tuning is also recommended when you want to incorporate your ~~comapy's~~ company's unique knowledge base. For ex., if you are aiming to use a model to draft responses to customer-support inquiries, fine-tuning on old conversations with customers can improve the quality of the o/p.

Step-by-Step Guide

Step ①: Prepare & Validate the dataset

We will work with the CoEDIT dataset of text editing examples. In each example, the user asks a writing assistant to rewrite text to suit a specific task (editing fluency, coherence, clarity, or style) & receive a response. Below you can see some examples from the raw dataset.

```
{"_id": "57241",
 "task": "coherence",
 "src": "make the text more coherent: It lasted for
         60 minutes. It featured the 3 men taking
         questions from a studio audience.",
 "tgt": "Lasting for 60 minutes, it featured the 3
         men taking questions from a studio audience."
}
```

↳ We will use the "src" & "tgt" from each example, which correspond to the user's prompt & the writing assistant's response respectively. Instead of using the full dataset, we will use a subset focused on making text coherent: 927 total conversations,

↳ To format the dataset for the Python SDK, we create a JSONL where each JSON object is a conversation containing a series of messages.

- A system message in the beginning, acting as the preamble that guides the whole conversation ~~containing a series of messages~~.

- multiple pairs of User and Chatbot messages, representing the conversation that takes place b/w a human user & a chatbot.

↳ For more details on best practices for formatting your dataset, check the documentation. Here is a preview of prepared dataset:

```
{ 'messages' :
  [ { 'role' : 'system',
      'content' : 'You are a writing assistant that helps the
                   user write coherent text'.

    },
    { 'role' : 'User',
      'content' : 'Make the text more coherent : It lasted
                   for 60 minutes.

    },
    { 'role' : 'Chatbot',
      'content' : 'Lasting for 60 minutes, it featured
                   the 3 men taking questions from a
                   studio audience.'

    }
  ]
}.
```

Fine-Tune the Model

→ We kick off a fine-tuning job by navigating to the fine-tuning tab of the Dashboard. Under "Chat", click on "Create a Chat model".

→ Next, upload the .Jsonl file you just created as the training set by clicking on the "TRAINING SET" button. When ready, click on "Review data" to proceed to next step.

→ Then, you'll see a preview of how the model will ingest your data. It shows the total no. of conversations, the total no. of turns, & the average no. of turns per conversation. If anything is wrong with data, the page will also provide suggested changes to fix the data file. Otherwise, if everything looks good, you can proceed to the next step.

→ Next, you'll see an estimated cost of fine-tuning, followed by a page where you'll provide a nickname to your model. We used "coedit-coherence" as the nickname for our model. This page also allows you to provide custom values for the hyperparameters used ~~at~~ during training, but we'll keep them at the default values for now.

→ Once you have filled in a name, click on "Start training" to kick off the fine tuning process. This will navigate you to a page where you can monitor the status of the model. A model that has finished fine-tuning will show the status as Ready.

↳ Step-③ Evaluate the Fine-Tuned Model

Once the model has completed the fine-tuning process, it's time to evaluate its performance.

↳ With Test data

When you're ready to use the fine-tuned model, navigate to API tab. There, you'll see the model ID that you should use when calling co.chat().

↳ In the following code we supply a message from the test dataset to both the pre-trained & the fine-tuned models for comparison.

```
user_message = "Make the text coherent : Pimelodella kronei is a
                species of 3-barbeled catfish endemic to Brazil,
                discovered by German - - - "

preamble = " You are a writing assistant that helps the user write!

# Get default model response.
response_pretrained = co.chat(
                message = user_message,
                preamble = preamble,
                )

# Get fine-tuned model response
response_fine tuned = co.chat(
                message = user_message,
                model = 'acb - - - - '
                preamble = preamble,
                )
print( f" Fine-tuned response: {response_fine tuned.text}"
```

In this example,
that are an improve
However, the fine-t
the style of th
succint:.

↳ In the Chat

We have demonst
good answers to
participant in long

To see this,
Building a Chatbo
The only change u
when using co.ch
with the mod

```
# Create a
import uuid
conversation
print ('st
while Tr
#
```

In this example, both models provide reasonable answers that are an improvement over the user's original text. However, the fine-tuned model's response better matches the style of the fine-tuning data, because it is more succint.

In the Chat context

We have demonstrated that the fine-tuned model can provide good answers to individual questions. But it is also a competent participant in longer, multi-turn conversations.

To see this, we will borrow from the code in the Building a Chatbot chapter to build a simple chat interface. The only change we need to make is to supply the model nickname when using co.chat_stream() to ensure that we are chatting with the model we just fine-tuned.

```
# Create a conversation ID
import uuid
conversation_id = str (uuid.uuid4())
print ('starting the chat. Type "quit" to end. \n")

while True:
        # User message
        message = input ("User: ")
        # Typing "quit" ends the conversation
        if message.lower() == 'quit':
                print ("Ending chat.")
                break
```

```
# chatbot response
stream = co.chat_stream(message= message,
        model = 'acb- - - ',
        preamble = preamble,
        conversation_id= conversation_id)
print("Chatbot: ", end=' ')
for event in stream:
    if event.event_type == "text-generation":
        print(event.text, end=' ')
print("\n", "-"*50)
```

We can use the above code to have a conversation with the fine-tuned model.

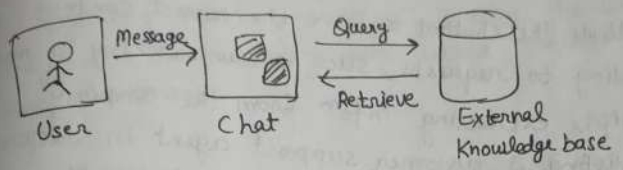
↳ Starting the chat. Type "quit" to end.

User : Hello
Chatbot: Hello, how can I help you today?

User: I'm fine. Can I ask you for help with some tasks?
Chatbot: I am happy to help you.

The model also did a good job with context switching; it can hold a conversation when the user switches from friendly greetings, to a request for writing help, to travel planning & finally back to writing assistance.

Introduction to RAG.



Here, you'll learn about connecting LLMs to external knowledge sources, enhancing the accuracy & relevance of chatbot responses.

We have so far explored LLM chatbots that only have access to the data they have been trained on, or their internal knowledge. In many app's, particularly for enterprise use, a chatbot needs to also be able to access external knowledge for it to be useful.

Cohere's Chat endpoint comes with a retrieval-augmented generation (RAG) feature that makes it possible to connect to external knowledge bases & deliver more accurate responses.